

Sources

Senior Challenge 3 - Vision AI target detection and follow

Everything in this document was checked directly against files present on the development machine - LICENSE files, `git remote -v`, `pip show`, embedded checkpoint metadata - during this session. Nothing here is copied from memory or assumed. Where a license or provenance claim could not be independently verified this way, it is flagged explicitly rather than asserted.

1 Reference repo: github.com/monemati/PX4-ROS2-Gazebo-YOLOv8

This repo was named in the original task brief as a reference. It was **never fetched or read** - the development machine had no internet access for essentially the entire project (confirmed repeatedly during the PX4/ROS 2 bridge debugging session; see `WRITEUP.md`). No code, config, launch file structure, gain values, or world file was copied from it, for the simple reason that it was never opened. Everything in `vision/`, `executor/`, `ros2_nodes/`, and `sim/` was independently written against PX4's own public ROS 2 API and Ultralytics' own public Python API (see below).

Action item: if internet access becomes available before submission, open that repo, check its license, and compare its approach. If anything concrete is consciously adopted after that review, this section should be updated with attribution. As of this writing, that comparison has not happened.

2 Direct dependencies

All verified via the actual checkout's LICENSE file and `git remote -v` on this machine, not from memory.

Project	License	Used for
PX4-Autopilot github.com/PX4/PX4-Autopilot	BSD 3-Clause	The SITL firmware itself (make <code>px4_sitl_gz_x500_mono_cam</code>). Checked out at main, commit <code>e0137fe7a7</code> plus this project's own commit <code>08bf7e70ac</code> on top.
<code>px4_msgs</code> github.com/PX4/px4_msgs	BSD 3-Clause	ROS 2 message definitions for the uXRCE-DDS bridge (<code>VehicleStatus</code> , <code>TrajectorySetpoint</code> , etc). Pinned to tag <code>v1.17.0</code> (commit <code>86d8239</code>) - load-bearing, not cosmetic; see below.

Project	License	Used for
ros_gz (includes ros_gz_image, ros_gz_bridge) github.com/gazebo-sim/ros_gz	Apache-2.0	Bridges the Gazebo camera topic into ROS 2 (<code>ros_gz_image_image_bridge</code>). Built from source in <code>~/px4_ros2_ws</code> against <code>GZ_VERSION=harmonic</code> ; the apt package (<code>ros-humble-ros-gz-image</code>) is linked against the wrong Gazebo generation.
Micro-XRCE-DDS-Agent github.com/eProsima/Micro-XRCE-DDS-Agent	Apache-2.0	The PX4 to ROS 2 DDS bridge (<code>MicroXRCEAgent udp4 -p 8888</code>).
PX4-gazebo-models github.com/PX4/PX4-gazebo-models	BSD 3-Clause	Submodule at <code>PX4-Autopilot/Tools/simulation/gz/</code> . Home of the stock <code>arucotag</code> model, whose <code>pbr/albedo_map</code> material pattern was reused for <code>target_person</code> , and now also of <code>target_person</code> itself, added this session (submodule commit <code>b8b7aa6</code>).

3 Ultralytics YOLOv8 (code and pretrained weights)

Code

The `ultralytics` PyPI package, version 8.4.93 in this project's venv. `pip show ultralytics` reports **License: AGPL-3.0** directly - not inferred, that is the literal metadata field. Every top-level module in the installed package (for example `ultralytics/utils/__init__.py`, `ultralytics/cfg/default.yaml`) carries the header `# Ultralytics AGPL-3.0 License - https://ultralytics.com/license` (the header in the actual source file also includes a rocket emoji character, omitted here for typesetting reasons).

Weights (`yolo8n.pt`)

The checkpoint's own embedded metadata, loaded via `torch.load` and inspected directly rather than assumed, reports:

```
date: 2022-12-30T00:12:08
version: 8.0.0.dev0
license: AGPL-3.0 License (https://ultralytics.com/license)
```

So the pretrained weights are self-declared AGPL-3.0 by Ultralytics themselves, the same as the code.

Flag: Ultralytics also sells an Enterprise license for commercial use outside AGPL-3.0's copyleft terms (referenced at ultralytics.com/license). This session had no internet access to check the current Enterprise-versus-AGPL boundary conditions or pricing, so confirm which applies to your actual use case. A take-home engineering exercise is very likely fine under AGPL-3.0 itself, but verify before any broader distribution.

API usage

```
from ultralytics import YOLO, model(frame), .boxes.{xyxy, conf, cls}, .names in vision/detector.py
```

is Ultralytics' own documented public Python API, not copied from any third party.

4 grace_hopper.jpg - the target_person detection texture

Where it came from

Vendored locally as part of matplotlib's own `mpl-data/sample_data/` (version 3.10.9 in this project's venv, at `venv/lib/python3.10/site-packages/matplotlib/mpl-data/sample_data/grace_hopper.jpg`). Chosen specifically because it required zero network fetch - the development machine has no internet access to pull a Fuel-hosted humanoid mesh (see `WRITEUP.md` and `README.md`, "Known limitations").

Matplotlib's own license

Verified via `pip show matplotlib` and the bundled `LICENSE` file: matplotlib's own permissive license (PSF-style, "License agreement for matplotlib versions 1.3.0 and later"), not AGPL or copyleft.

The photo's own copyright status - flagged, not verified

This appears to be Grace Hopper's official U.S. Navy portrait. Photographs taken by U.S. federal employees or military personnel as part of official duties are typically public domain in the United States under 17 U.S.C. §105 (no copyright protection for U.S. government works), and matplotlib ships it with no separate attribution or license file alongside it - `mpl-data/sample_data/README.txt` was checked directly and only says "sample data needed for some of matplotlib's examples," with no per-file attribution. The "official Navy portrait / public domain" claim could not be independently confirmed against an authoritative primary source given no internet access this session. **Verify this yourself** before treating it as cleared for anything beyond this internal engineering and simulation use.

How it was verified detectable before use

Not just assumed: before committing to it as the texture, this project's own `vision.detector.Detector`, the exact class the live mission uses, was run against the raw JPEG directly - `person` at 0.91 confidence, a single clean detection. Only after that check did it get wrapped into the `target_person` Gazebo model (see `WRITEUP.md` for the bug-fixing narrative and `PX4-Autopilot/Tools/simulation/gz/models/target_person` for the in-repo attribution note).

5 What this module is actually built on

Design and API patterns, not copied code.

- **PX4 ROS 2 offboard control pattern** (/fmu/in/offboard_control_mode, /fmu/in/trajectory_setpoint, /fmu/in/vehicle_command, /fmu/out/vehicle_status_v1, /fmu/out/vehicle_local_position_v1; the more-than-2Hz OffboardControlMode heartbeat requirement; the arm/offboard-engage VehicleCommand sequence; the best-effort/transient-local QoS profile needed to talk to the uXRCE-DDS bridge) is PX4's own publicly documented ROS 2 API. This was not just read from docs but live-verified against a real running PX4 instance this session - see WRITEUP.md for the debugging record (px4_msgs version mismatch, NAV_DLL_ACT arming block).
- **ros_gz_image's image_bridge CLI usage** (positional Gazebo topic argument, ros-args -r <topic>:=<new-name> remap) is that project's own documented CLI syntax.
- **The proportional visual-servoing control law** in vision/follow_controller.py (bounding-box offset to yaw/vertical rate, bounding-box size to forward speed, EMA smoothing, dead-bands, lost-target timeout) was derived from scratch for this challenge - standard image-based visual servoing theory, not copied from any particular repo. See that file's module docstring for the full derivation.
- **PX4 arming and health-check internals** (NAV_DLL_ACT, rcAndDataLinkCheck.cpp, the airframe .post override mechanism in rcS) were root-caused by reading PX4-Autopilot's own source during this session's debugging, not from any external doc or reference repo - see WRITEUP.md for the full account.

6 What was investigated as a hypothesis and ruled out

Worth recording honestly: during the original bridge-debugging session, domain-ID mismatch - PX4's uXRCE-DDS client or the Micro-XRCE-DDS-Agent using a different DDS domain than the ROS 2 side's ROS_DOMAIN_ID - was considered as a candidate explanation for "zero messages received." It was checked and **ruled out**, not confirmed as a bug: MicroXRCEAgent help shows discovery defaults to port 7400 (domain 0's SPDP port) regardless of ROS_DOMAIN_ID, but in this environment ROS_DOMAIN_ID was already unset (defaulting to 0) on every terminal, so there was no actual mismatch to fix. The real root causes of that symptom were the px4_msgs version mismatch and, separately, two dead PX4/Agent processes - both documented in WRITEUP.md. It is included here only as an honest record of what was checked, not as a claimed fix.

7 Other docs consulted during debugging

No code copied, used for orientation only.

- PX4's own in-repo source as the primary reference during live debugging: src/modules/commander/HealthAndSafety.cpp (root cause of the NAV_DLL_ACT arming block), ROMFS/px4fmu_common/init.d-posix (the .post autostart-override mechanism), Tools/simulation/gz/models/arucotag/model.sdf (the pbr/albedo_map texture pattern copied for target_person).
- Standard Linux and DDS tooling docs referenced only for command syntax during debugging (ldd, gz topic/gz service CLI, pymavlink's mavutil.mavlink_connection API) - no text or code copied, just used as intended by their own public interfaces.

8 Action items before treating this as final

1. If internet access becomes available, open `github.com/monemati/PX4-ROS2-Gazebo-YOLOv8`, read its license, and note here whether anything was adopted after comparison. Nothing has been, as of this writing.
2. Verify the Grace Hopper portrait's public-domain status independently if `target_person/grace_hopper.jpg` will be used beyond this internal engineering exercise (see the flagged section above).
3. Confirm which Ultralytics license tier - AGPL-3.0 or Enterprise - actually applies to your intended use of `yolov8n.pt` if this goes beyond a take-home exercise.